

From Arrays to Accessibility

A ground-up tour of Java and Kotlin, taught entirely through the Android app sitting in `F:\NoShorts` — the one that closes YouTube Shorts. By the end you should be able to read every line of it, and fix the bug that is still in it.

Written for Chandradithya Janaswami
Assumes You can use a terminal and git. No Java, no Kotlin, no Android.
Project github.com/JV SCHANDRADITHYA/NoShorts
Language level Kotlin 1.9 · JDK 17 · Android SDK 34

CONTENTS

- | | | | |
|----|---------------------------------|----|---|
| 01 | What you are actually running | 06 | Lambdas — the one that unlocks the codebase |
| 02 | Values, types, and yes — arrays | 07 | Java ↔ Kotlin, side by side |
| 03 | Functions and control flow | 08 | How Android actually works |
| 04 | Null, and why Kotlin exists | 09 | NoShorts, line by line |
| 05 | Classes, objects, inheritance | 10 | The bug that is still in there |

PART 01

What you are actually running

Before any syntax, get the layers straight. Almost every beginner confusion about Java and Android is really a confusion about *which layer* a thing belongs to.

The JVM, and why it matters

Java source code (`.java`) does not compile to machine code the way C does. It compiles to **bytecode** (`.class`), an instruction set for an imaginary computer called the **Java Virtual Machine**. At runtime the JVM turns that bytecode into real instructions for whatever CPU it is on. That is the "write once, run anywhere" pitch.

Kotlin is a different language that compiles to the *same bytecode*. That single fact explains most of Kotlin's design: it can call any Java library, Java can call it back, and it inherits Java's whole ecosystem for free. Kotlin is not a layer on top of Java — it is a sibling language sharing a target.

Android does not run a standard JVM. It runs **ART** (Android Runtime), which takes that same bytecode and compiles it to native ARM code when your app installs. Different engine, same fuel.

CONCEPT

You write Kotlin → the Kotlin compiler emits JVM bytecode → Android's build tools repackage it into `classes.dex` → ART runs it on the phone. Every language feature you are about to learn has to survive that chain, which is why some Kotlin features are "just" clever compiler tricks over plain Java constructs.

What an APK is

An `.apk` is a ZIP file. Rename it and open it. Inside NoShorts you will find exactly what we saw when verifying the build:

<code>classes.dex</code>	All your compiled Kotlin, converted to Dalvik bytecode. Ours is ~2 MB, almost all Kotlin's standard library.
<code>AndroidManifest.xml</code>	The app's declaration of itself: package name, components, permissions. Android reads this before running a single line of your code.
<code>resources.arsc</code>	Compiled resources — strings, colours, layouts — looked up by integer ID at runtime.
<code>META-INF/</code>	The signature. Android refuses to install an update signed by a different key than the installed version. This is exactly the wall we hit when CI generated a fresh debug key each run.

Gradle, the build system

Gradle is the thing that orchestrates: download dependencies, run the Kotlin compiler, compile resources, package, sign. You configure it in `build.gradle.kts` — which is itself written in Kotlin.

`app/build.gradle.kts` — the whole build config

```
android {
    namespace = "com.noshorts.blocker" // where the generated R class lands
    compileSdk = 34                     // API level you compile against

    defaultConfig {
        applicationId = "com.noshorts.blocker" // unique ID on the device
        minSdk = 24 // oldest Android you support
        targetSdk = 34 // newest you have tested on
        versionCode = 1 // integer, must increase
        versionName = "1.0" // what humans see
    }
}
```

GOTCHA

`applicationId` is the app's identity on the device — two apps with the same one cannot coexist. `namespace` is a compile-time concept only. They are usually identical, which hides the difference until the day it bites you.

Values, types, and yes — arrays

You said you cannot build an array in Java. Let us kill that one first, then explain why you will almost never write one in Kotlin anyway.

Declaring a value

Java puts the type first. Kotlin puts a keyword first and usually infers the type.

JAVA	KOTLIN	NOTES
<code>int count = 5;</code>	<code>var count = 5</code>	Reassignable. Kotlin infers <code>Int</code> .
<code>final int MAX = 10;</code>	<code>val MAX = 10</code>	<code>val</code> = cannot be reassigned.
<code>String s = "hi";</code>	<code>val s: String = "hi"</code>	Explicit type is optional.

Default to `val`. Reach for `var` only when something genuinely changes over time. In our service, `backAttempts` is a `var` because it counts up; `PLAYER_ID_PREFIXES` is a `val` because the list never changes.

The core types

Java splits types into **primitives** (`int`, `long`, `double`, `boolean`, `char`) — raw machine values — and **objects** (`String`, `Integer`, everything else). Kotlin presents everything as an object (`Int`, `Long`, `Boolean`) and quietly compiles down to primitives where it can.

Kotlin numeric and text basics

```
val whole    = 42           // Int      (32-bit)
val big      = 3_000_000_000L // Long    (64-bit, note the L)
val fraction = 2.5          // Double
val yes      = true         // Boolean
val letter   = 'A'          // Char    (single quotes)
val text     = "hello"      // String  (double quotes)

// Underscores are just visual separators the compiler ignores:
val settle = 900L           // we use this for ESCAPE_SETTLE_MS
val quiet  = 5_000L         // reads better than 5000L
```

String templates beat concatenation. A `$` inlines a variable; `${...}` inlines an expression:

String templates

```
val n = 3
println("Back failed $n times")           // Back failed 3 times
println("Half of that is ${n / 2}")       // Half of that is 1
```

GOTCHA

In `ShortsBlockerService.kt` I deliberately wrote `"Back failed " + backAttempts + " times"` instead of a template. Not style — I was generating those files through shell heredocs, where a bare `$` gets eaten by the shell before it reaches the file. In your own editor, always use the template form.

Arrays — the thing you asked about

An array is a fixed-length block of slots, all the same type, indexed from 0. Fixed-length is the key word: once created, the size never changes.

Java

```
// 1. Empty array of a given size — every slot starts at 0
int[] numbers = new int[5];
numbers[0] = 42;
numbers[4] = 7;           // last valid index is 4, not 5

// 2. Literal, size inferred from the contents
int[] primes = {2, 3, 5, 7, 11};

// 3. Reading
System.out.println(primes[2]);    // 5
System.out.println(primes.length); // 5 — a field, no parentheses

// 4. Looping
for (int i = 0; i < primes.length; i++) {
    System.out.println(primes[i]);
}
for (int p : primes) {            // enhanced for, when you don't need the index
    System.out.println(p);
}

// 5. Array of objects
String[] names = new String[3];   // all three slots start as null
names[0] = "Adithya";
```

Kotlin — the same five things

```
val numbers = IntArray(5)          // zero-filled
numbers[0] = 42

val primes = intArrayOf(2, 3, 5, 7, 11)

println(primes[2])                 // 5
println(primes.size)               // 5 — a property, still no ()

for (i in primes.indices) println(primes[i])
for (p in primes) println(p)

val names = arrayOfNulls<String>(3) // Array<String?>
```

GOTCHA

Index out of bounds is the classic crash. An array of size 5 has indices `0..4`. Writing `numbers[5]` compiles fine and throws `ArrayIndexOutOfBoundsException` at runtime. The compiler cannot save you here — it does not know the index until the program runs.

Why you will rarely write an array

Fixed length is a pain. In practice you use **List**, which grows, and which Kotlin gives beautiful syntax for. This is what our code actually uses:

`ShortsSignals.kt` — a real List from the project

```
val PLAYER_ID_PREFIXES = listOf(
    "reel_recycler",
    "reel_player",
    "reel_watch",
    "reel_dyn",
    "reel_progress_bar"
)
```

YOU WANT

KOTLIN

Read-only list	<code>listOf("a", "b")</code>
Growable list	<code>mutableListOf<String>()</code> then <code>.add(x)</code>
Key → value map	<code>mapOf("a" to 1)</code>
Unique values	<code>setOf(1, 2, 3)</code>
Length	<code>.size</code> for all of them

The `<String>` part is a **generic** — it tells the compiler what the list holds, so `masks.add(view)` only accepts a `View`, and reading an element gives you a `View` back with no casting. In our service: `private val masks = mutableListOf<View>()`.

Functions and control flow

Functions

Anatomy of a Kotlin function

```
fun add(a: Int, b: Int): Int {
    return a + b
}

// ^      ^      ^      ^
// |      |      |      |  return type
// |      |      |      |  parameter: name first, then type
// |      |      |      |  function name
// |      |      |      |  the keyword, always

// Single-expression form – return type inferred:
fun add(a: Int, b: Int) = a + b

// Returns nothing? The type is Unit, and you omit it:
fun greet(name: String) {
    println("Hi $name")
}
```

Java writes the return type first and has no `fun` keyword: `int add(int a, int b) { return a + b; }`. Java's "returns nothing" is `void`; Kotlin's is `Unit`.

Kotlin also gives you **default arguments**, which is why Java's habit of writing five overloads of the same method mostly disappears:

Default and named arguments

```
fun log(message: String, level: String = "INFO") {
    println("[$level] $message")
}

log("service connected")           // [INFO] service connected
log("back failed", level = "WARN") // [WARN] back failed
```

if, when, loops

In Kotlin `if` is an *expression* — it produces a value, so there is no ternary `?:` operator:

Control flow

```
// if as an expression
val label = if (serviceOn) "Active" else "Not active"

// when – like switch, but far better. This is from MainActivity:
linkGroup.check(
    when (prefs.linkMode) {
        Prefs.LINK_IN_BROWSER → R.id.link_browser
        Prefs.LINK_DISCARD   → R.id.link_discard
        else                  → R.id.link_video
    }
)

// when with conditions instead of values
val size = when {
    n < 10    → "small"
    n < 100   → "medium"
    else      → "large"
}

// loops
for (i in 0 until node.childCount) { } // 0,1,..,childCount-1 ← we use this
for (i in 1..5) { }                  // 1,2,3,4,5 (inclusive!)
for (rect in rects) { }              // each element
while (queue.isEmpty()) { }          // ← and this
```

GOTCHA

`0..5` includes 5. `0 until 5` stops at 4. When walking an array or a child list you almost always want `until`, which is why `forEachNode` uses `0 until node.childCount`. Getting this wrong is the Kotlin off-by-one.

Null, and why Kotlin exists

In Java any object reference can be `null`, and calling a method on `null` throws `NullPointerException`. Its inventor called it his billion-dollar mistake. Kotlin's single biggest feature is making the compiler track it.

By default a Kotlin type **cannot** hold null. Add `?` to allow it — and then the compiler forces you to handle it.

Nullability

```
var a: String = "hello"
a = null           // COMPILE ERROR – not a crash, a refusal to build

var b: String? = "hello"
b = null           // fine
println(b.length)  // COMPILE ERROR – b might be null, handle it
```

THE FOUR TOOLS

Handling nullable values – all four appear in NoShorts

```
// 1. ?. safe call – returns null instead of crashing
val n = b?.length           // Int? (null if b is null)
wm?.removeView(view)        // ← clearMasks() does exactly this

// 2. ?: elvis – supply a fallback (tilt your head: it has a quiff)
val len = b?.length ?: 0     // Int, never null

// 3. ?: return – bail out early. THE most common line in Android code:
val root = rootInActiveWindow ?: return // ← onAccessibilityEvent, line 1

// 4. !! assert not-null – crashes if wrong. Avoid. We never use it.
val forced = b!!.length
```

That third form deserves attention because it appears at the top of our event handler. Read it as: "give me `rootInActiveWindow`; if it is null, abandon this function right now." After that line, the compiler knows `root` is non-null and stops nagging.

lateinit

Sometimes a value genuinely cannot be set at construction time. Android does not let you initialise things in a service's constructor — the framework has not finished wiring it up. So:

ShortsBlockerService.kt

```
private lateinit var prefs: Prefs           // "I promise to set this before use"

override fun onServiceConnected() {
    super.onServiceConnected()
    prefs = Prefs(this)                     // the promise, kept
}
```


GOTCHA

`lateinit` switches the null check off and trades it for `UninitializedPropertyAccessException` if you touch it too early. It is a promise you make to the compiler. If NoShorts ever crashes the instant the service starts, a broken `lateinit` promise is the first thing to suspect.

Classes, objects, inheritance

A **class** is a blueprint. An **object** (or instance) is one thing built from that blueprint. `Prefs` is a class; `Prefs(this)` builds one.

A class with a constructor

```
class Dog(val name: String, var age: Int) {

    fun bark() {
        println("$name says woof")
    }
}

val rex = Dog("Rex", 3)           // no `new` keyword in Kotlin
rex.bark()                       // Rex says woof
rex.age = 4                      // allowed: age is a var
rex.name = "Max"                 // ERROR: name is a val
```

Those constructor parameters marked `val` / `var` become properties automatically. The Java equivalent is roughly fifteen lines of fields, a constructor, and getters. This is the single biggest reason Kotlin files are shorter.

Properties with custom behaviour

A property does not have to be storage. It can be a pair of functions wearing the costume of a variable — which is exactly how `Prefs` works:

`Prefs.kt` — the real thing

```
class Prefs(context: Context) {

    private val sp: SharedPreferences =
        context.applicationContext.getSharedPreferences("noshorts", Context.MODE_PRIVATE)

    var blockPlayer: Boolean
        get() = sp.getBoolean(KEY_BLOCK_PLAYER, true)
        set(value) = sp.edit().putBoolean(KEY_BLOCK_PLAYER, value).apply()
}
```

Nothing is stored in the `Prefs` object at all. Writing `prefs.blockPlayer = false` looks like assigning a field, but it runs the setter, which writes to disk. Reading it runs the getter, which reads from disk. That is why the service and the settings screen always agree — they are both talking to the same file, not to a shared variable.

CONCEPT

`context` has no `val` in front of it, so it is a constructor parameter only — usable during construction, not kept as a property. `sp` is a property, and `private` means nothing outside the class can touch it. Deliberate: callers get the typed accessors, never the raw preferences file.

Inheritance and override

Inheritance lets a class extend another, keeping its behaviour and replacing parts. This is the entire basis of Android app development: the framework defines base classes, you subclass them and fill in the blanks.

The shape of every Android component you will ever write

```
class ShortsBlockerService : AccessibilityService() {  
  
    override fun onAccessibilityEvent(event: AccessibilityEvent?) {  
        // your code – the framework calls this, you never do  
    }  
  
    override fun onServiceConnected() {  
        super.onServiceConnected() // let the parent do its setup first  
        prefs = Prefs(this)  
    }  
}
```

<code>:</code> <code>AccessibilityService()</code>	"extends". The parentheses call the parent's constructor. Java writes <code>extends AccessibilityService</code> .
---	---

<code>override</code>	Mandatory in Kotlin — you must state that you are replacing a parent method. Java's <code>@Override</code> is optional, so typos silently create a brand-new method that nothing ever calls.
-----------------------	--

<code>super.x()</code>	Calls the parent's version. Skip it in a lifecycle method and you will break framework bookkeeping in ways that are miserable to debug.
------------------------	---

CONCEPT – INVERSION OF CONTROL

You never call `onAccessibilityEvent`. Android does, whenever something happens in YouTube. Your job is to *be available*. Nearly all Android code is callbacks the system invokes on your behalf — which is why "where does the program start?" has no clean answer here.

object and companion object

Kotlin has a keyword for "there is exactly one of these, ever":

Two flavours of singleton, both used in NoShorts

```
// A standalone singleton. Never instantiated – you just use it.
object ShortsSignals {
    const val YOUTUBE_PACKAGE = "com.google.android.youtube"
    val PLAYER_ID_PREFIXES = listOf("reel_recycler", "reel_player")

    fun matches(id: String?, prefixes: List<String>): Boolean { ... }
}

ShortsSignals.matches(x, ShortsSignals.PLAYER_ID_PREFIXES)

// A singleton attached to a class – Kotlin's replacement for `static`.
class ShortsBlockerService : AccessibilityService() {
    companion object {
        private const val TAG = "NoShorts"
        private const val ESCAPE_SETTLE_MS = 900L
    }
}
```

Constants live in a `companion object` because there is no reason for every instance to carry its own copy of the number 900. `const val` goes one step further and inlines the value at compile time — free, but only legal for primitives and strings.

Lambdas — the one that unlocks the codebase

If one concept stops you reading `ShortsBlockerService.kt`, it is this. A **lambda** is a function you pass around like a value — a block of code handed to something else to call later.

Lambda syntax, built up

```
// A lambda is code in braces:
val greet = { println("hi") }
greet()                                // hi

// With parameters — name them before the →
val add = { a: Int, b: Int → a + b }
println(add(2, 3))                     // 5

// One parameter can go unnamed; it is called `it`
val double = { n: Int → n * 2 }
val double2: (Int) → Int = { it * 2 }  // identical
```

Functions that take functions

A parameter can have a *function type*. `(View) → Boolean` means "takes a View, returns a Boolean".

`ShortsBlockerService.kt` — our tree walker

```
private fun forEachNode(
    root: AccessibilityNodeInfo,
    visitor: (AccessibilityNodeInfo) → Boolean    // ← a function parameter
) {
    val queue = ArrayDeque<AccessibilityNodeInfo>()
    queue.addLast(root)
    var visited = 0
    while (queue.isNotEmpty() && visited < MAX_NODES) {
        val node = queue.removeFirst()
        visited++
        if (!visitor(node)) return                // ← call the passed-in function
        for (i in 0 until node.childCount) {
            val child = node.getChild(i)
            if (child != null) queue.addLast(child)
        }
    }
}
```

And here is how it gets used. When a lambda is the *last* argument, Kotlin lets you move it outside the parentheses — the "trailing lambda" — which is why so much Kotlin looks like it has custom syntax:

The same function, two jobs

```
private fun hasAny(root: AccessibilityNodeInfo, prefixes: List<String>): Boolean {
    var found = false
    forEachNode(root) { node → // ← trailing lambda
        if (ShortsSignals.matches(node.viewIdResourceName, prefixes)) {
            found = true
            false // last expression = return value
        } else {
            true // keep walking
        }
    }
    return found
}
```

CONCEPT – IMPLICIT RETURN

A lambda has no `return` statement. Its value is whatever its **last expression** evaluates to. Above, the bare `false` and `true` are the return values — they look like stray words, and they are the whole control mechanism: `false` means "stop walking", `true` means "keep going".

The payoff: `forEachNode` knows how to walk a view tree and nothing else. `hasAny`, `collectRects` and `logViewIds` all reuse it with different lambdas. Write the traversal once, vary the behaviour.

Lambdas as event handlers

`MainActivity.kt` – the Android idiom

```
enableButton.setOnClickListener {
    startSettings(Intent(Settings.ACTION_ACCESSIBILITY_SETTINGS))
}

private fun bindSwitch(id: Int, initial: Boolean, onChange: (Boolean) → Unit) {
    val view = findViewById<Switch>(id)
    view.isChecked = initial
    view.setOnCheckedChangeListener { _, checked → onChange(checked) }
}

bindSwitch(R.id.switch_player, prefs.blockPlayer) { prefs.blockPlayer = it }
```

Read that last line: "wire up the switch with id `switch_player`, start it in the state currently saved in preferences, and when it changes, write the new value (`it`) back." The underscore in `{ _, checked → }` means "this parameter exists but I do not care about it".

Java ↔ Kotlin, side by side

You will meet Java constantly — every Android tutorial, every StackOverflow answer over five years old. Here is the translation table.

JAVA	KOTLIN	
<code>final String s = "x";</code>	<code>val s = "x"</code>	immutable
<code>String s = "x";</code>	<code>var s = "x"</code>	mutable
<code>new Dog("Rex")</code>	<code>Dog("Rex")</code>	no new
<code>void f()</code>	<code>fun f()</code>	returns Unit
<code>int f()</code>	<code>fun f(): Int</code>	type last
<code>extends / implements</code>	<code>:</code>	both
<code>static final int X</code>	<code>const val X in companion object</code>	
<code>arr.length</code>	<code>arr.size</code>	list too
<code>a == b (objects)</code>	<code>a === b</code>	same instance
<code>a.equals(b)</code>	<code>a == b</code>	same value
<code>switch</code>	<code>when</code>	no fallthrough
anything can be null	<code>?</code> marks nullable	compiler-checked
semicolons required	semicolons optional	omit them

GOTCHA — THE EQUALITY TRAP

Java's `==` on objects compares *identity*, so comparing two Strings with `==` is a famous bug. Kotlin flips it: `==` compares *value* (it calls `equals` for you, null-safely) and `===` compares identity. Coming from Java, you must unlearn this one.

Where our code compares `rects == maskedRects`, that is Kotlin `==` — a genuine element-by-element list comparison. It is what stops us tearing down and rebuilding the overlays on every single accessibility event.

How Android actually works

The manifest is the app's contract

Android reads `AndroidManifest.xml` before executing any of your code. Nothing exists unless declared here.

`AndroidManifest.xml` — our service declaration

```
<service
  android:name=".ShortsBlockerService"
  android:exported="false"
  android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE">
  <intent-filter>
    <action android:name="android.accessibilityservice.AccessibilityService" />
  </intent-filter>
  <meta-data
    android:name="android.accessibilityservice"
    android:resource="@xml/accessibility_service_config" />
</service>
```

<code>android:name</code>	The class. A leading dot means "relative to the package".
<code>exported="false"</code>	No other app can start it. Only the system.
<code>permission</code>	Whoever binds must hold this permission — in practice, only Android itself.
<code>intent-filter</code>	How the system discovers you. This one says "I am an accessibility service", which is what makes NoShorts appear in the Accessibility settings list.
<code>meta-data</code>	Points at the config XML holding the real settings.

The config that scopes our privacy claim

`res/xml/accessibility_service_config.xml`

```
<accessibility-service
  android:accessibilityEventTypes="typeWindowStateChanged|typeWindowContentChanged"
  android:accessibilityFlags="flagReportViewIds|flagIncludeNotImportantViews"
  android:canRetrieveWindowContent="true"
  android:notificationTimeout="100"
  android:packageNames="com.google.android.youtube" />
```

`packageNames` is the important line and the reason the README's privacy claim is honest rather than a promise: the *framework* refuses to deliver events from any other app. Not a filter in our code that could have a bug — a restriction enforced above us. `flagReportViewIds` is equally load-bearing: without it, `viewIdResourceName` comes back null and the entire detection strategy silently stops working.

Resources and the R class

Anything not code — strings, layouts, colours, icons — lives in `res/`. At build time Android generates a class called `R` holding an integer constant for every resource.

The round trip

```
<!-- res/values/strings.xml -->
<string name="status_active">Active - Shorts are being blocked.</string>

<!-- res/layout/activity_main.xml -->
<TextView android:id="@+id/status_text" ... />
```

```
statusText = findViewById(R.id.status_text)    // @+id/ → R.id.
statusText.text = getString(R.string.status_active)
```

`@+id/` means "create this ID"; `@string/` means "look this one up". Keeping text in `strings.xml` is not bureaucracy — it is what makes translation possible, and it is why `getString(R.string.status_blocked, count)` can substitute `%1$d` for you.

The main thread

Android draws everything on one thread. Block it and the UI freezes; block it for ~5 seconds and the system kills you. Your accessibility callbacks arrive on it.

That is why `forEachNode` is capped at `MAX_NODES = 2500`: a runaway walk of a huge view tree would stall the phone. And it is why delayed work goes through a `Handler`:

Scheduling work for later, on the main thread

```
private val handler = Handler(Looper.getMainLooper())

handler.postDelayed(escapeCheck, ESCAPE_SETTLE_MS)    // run in 900 ms
handler.removeCallbacks(escapeCheck)                  // cancel if not yet run
```

`Looper.getMainLooper()` is the main thread's message queue. `postDelayed` puts a task on it with a timestamp. There is no `sleep` anywhere in Android UI code — you schedule and return.

CONCEPT — RUNNABLE

`escapeCheck` is an object : `Runnable` — an anonymous class created inline, implementing the one-method interface `Runnable`. Written as an object rather than a lambda because it needs to re-post *itself* (`handler.postDelayed(this, ...)`), and a lambda has no name to refer to.

NoShorts, line by line

Everything so far, assembled. The app is five Kotlin files and about 1,000 lines total.

FILE	JOB
ShortsSignals.kt	The view-id fingerprints. The only file that needs editing when YouTube changes.
Prefs.kt	Typed wrapper over SharedPreferences.
ShortsBlockerService.kt	All the actual behaviour.
LinkRedirectActivity.kt	Rewrites <code>/shorts/<id></code> to <code>/watch?v=<id></code> .
MainActivity.kt	The settings screen.

Detection

We cannot see YouTube's code. What we can see is the accessibility tree — the same structure a screen reader uses. Every view in it may carry a `viewIdResourceName` like `com.google.android.youtube:id/reel_recycler`. YouTube's Shorts views are internally called "reels", and that naming is the hook the whole app hangs on.

ShortsSignals.kt

```
fun matches(viewIdResourceName: String?, prefixes: List<String>): Boolean {
    if (viewIdResourceName == null) return false
    val leaf = viewIdResourceName.substringAfterLast('/')
    return prefixes.any { leaf.startsWith(it) }
}
```

`substringAfterLast('/')` turns the full id into just `reel_recycler`. `prefixes.any { ... }` is a standard-library function taking a lambda: "is this true for at least one element?" Prefix matching rather than exact matching is deliberate — YouTube renames leaf views between releases far more often than it renames the whole `reel_*` family.

The event handler

ShortsBlockerService.kt — the entry point for everything

```
override fun onAccessibilityEvent(event: AccessibilityEvent?) {
    if (event == null) return
    if (event.packageName != ShortsSignals.YOUTUBE_PACKAGE) return

    if (escaping) return // an escape is already in flight

    val root = rootInActiveWindow ?: return

    if (prefs.logViewIds) logViewIds(root)

    if (prefs.blockPlayer && hasAny(root, ShortsSignals.PLAYER_ID_PREFIXES)) {
```

```

        escapeShorts()
        return
    }

    quietUntil = 0L

    if (prefs.hideShelves) {
        updateShelfMasks(root)
    } else {
        clearMasks()
    }
}

```

Read it as a sequence of guards, each cheaply rejecting work before the expensive tree walk. This method runs many times per second while you scroll YouTube, so ordering matters.

The escape

This is the part we have already had to fix once, and the part still misbehaving.

ShortsBlockerService.kt – press once, then verify

```

private fun escapeShorts() {
    if (escaping) return
    if (SystemClock.uptimeMillis() < quietUntil) return

    escaping = true
    backAttempts = 1

    clearMasks()
    performGlobalAction(GLOBAL_ACTION_BACK)
    prefs.blockedCount = prefs.blockedCount + 1

    handler.postDelayed(escapeCheck, ESCAPE_SETTLE_MS)
}

private val escapeCheck = object : Runnable {
    override fun run() {
        val root = rootInActiveWindow
        val stillInShorts = root != null &&
            root.packageName == ShortsSignals.YOUTUBE_PACKAGE &&
            hasAny(root, ShortsSignals.PLAYER_ID_PREFIXES)

        if (!stillInShorts) {
            escaping = false
            backAttempts = 0
            return
        }

        if (backAttempts ≥ MAX_BACK_ATTEMPTS) {
            if (prefs.exitAppFallback) {
                performGlobalAction(GLOBAL_ACTION_HOME)
            } else {
                quietUntil = SystemClock.uptimeMillis() + QUIET_PERIOD_MS
            }
            escaping = false
            backAttempts = 0
            return
        }
    }
}

```

```

    }

    backAttempts++
    performGlobalAction(GLOBAL_ACTION_BACK)
    handler.postDelayed(this, ESCAPE_SETTLE_MS)
  }
}

```

`performGlobalAction(GLOBAL_ACTION_BACK)` is the only real power an accessibility service has here — it presses the same Back button you would. It cannot reach into YouTube; it can only drive the device.

The `escaping` flag is a **state machine**, and it exists because the first version did not have one. That version re-fired Back on every matching event with only a 350 ms debounce, and since YouTube keeps reel views in the tree throughout its exit animation, press two popped the screen *behind* Shorts and press three triggered the "Back isn't working" path straight to HOME. Hence: press once, wait, look again.

The overlays

An accessibility service cannot hide another app's views. So to remove Shorts shelves from the feed we draw our own black windows on top of them, positioned using the bounds the tree reports.

Why the flags matter more than the code

```

WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE or
WindowManager.LayoutParams.FLAG_NOT_TOUCHABLE or
WindowManager.LayoutParams.FLAG_LAYOUT_NO_LIMITS or
WindowManager.LayoutParams.FLAG_LAYOUT_IN_SCREEN

```

`or` here is not boolean "or" — it is **bitwise or**, combining flags into one integer. Each flag is a distinct bit; combining them sets several bits at once. Java writes this as `|`.

`FLAG_NOT_TOUCHABLE` is the design decision in the whole feature: touches pass straight through the mask, so the feed still scrolls normally underneath. An opaque, touchable overlay would hide the shelf and trap your finger.

GOTCHA — THE WATCHDOG

Because `packageName` restricts us to YouTube, we stop receiving events the moment you leave it — and our overlays would hang on screen forever. `maskWatchdog` re-posts itself every 800 ms, checks whether YouTube is still foreground, and tears the masks down when it is not. Every drawn overlay needs an owner responsible for removing it.

The bug that is still in there

Real code, real defect, still unfixed at the time of writing. Reported symptom: *opening Shorts closes the Short, then backs out of YouTube, then minimises it*. The state machine above was supposed to fix that and did not.

The prime suspect

Compare these two, carefully

```
// hasAny – decides whether to press Back
if (ShortsSignals.matches(node.viewIdResourceName, prefixes)) {
    found = true
    false
}

// collectRects – decides where to draw overlays
if (ShortsSignals.matches(node.viewIdResourceName, prefixes) && node.isVisibleToUser) {
    ...
}
```

`collectRects` checks `isVisibleToUser`. `hasAny` does not.

Android's bottom-navigation tabs are typically backed by fragments that stay alive after you leave them. If YouTube keeps the Shorts fragment in the tree once you are back on the Home tab, then `hasAny` keeps returning `true` for views that are no longer on screen. The service concludes it is still in Shorts, presses Back again, pops YouTube's own screens, and eventually walks the user out of the app — with no HOME action ever fired, which matches the empty log exactly.

The fix to try

Require the match to be visible *and* large. The full-screen Shorts player fills the display; a cached off-screen fragment does not.

Sketch – not yet applied

```
private fun isShortsPlayerOnScreen(root: AccessibilityNodeInfo): Boolean {
    val screen = Rect()
    root.getBoundsInScreen(screen)
    val minHeight = screen.height() * 0.6    // player fills most of the display

    var found = false
    forEachNode(root) { node →
        if (node.isVisibleToUser &&
            ShortsSignals.matches(node.viewIdResourceName, ShortsSignals.PLAYER_ID_PREFIXES)) {
            val b = Rect()
            node.getBoundsInScreen(b)
            if (b.height() ≥ minHeight) { found = true; return@forEachNode false }
        }
        true
    }
    return found
}
```

`return@forEachNode false` is a **labelled return**: plain `return` inside a lambda would try to exit the enclosing function, so Kotlin makes you name which one you mean.

How to confirm it before writing code

Do not guess — including when I guess. Get the ground truth:

1. In NoShorts, turn **off** "Close the Shorts player" so it cannot eject you mid-capture.
2. Open YouTube → Shorts, leave it on screen.
3. `adb shell uiautomator dump /sdcard/ui.xml` then `adb pull /sdcard/ui.xml`.
4. Press Back to the Home feed and dump again.
5. Diff the two for `reel_` ids. If they appear in *both*, the theory is confirmed.

That is the actual job: form a hypothesis, find the cheapest observation that could falsify it, then change one thing. The Kotlin is the easy part.